# Securing n8n MCP Server with Descope on lovable.dev

The Model Context Protocol (MCP) provides a standardized way for AI agents to discover and interact with external tools. By exposing n8n workflows as MCP tools, you can grant AI assistants like Claude the ability to execute complex automation tasks. However, directly exposing n8n webhook URLs creates significant security risks, as they can become targets for unauthorized access and abuse.

This guide provides a complete manual for building a secure, enterprise-grade architecture that leverages Descope's Agentic Identity Hub to protect your n8n tool backends. We will use lovable.dev to generate and deploy a Next.js MCP server, which will act as a secure gateway. This server will authenticate and authorize incoming requests from AI agents before securely triggering the appropriate n8n webhook. Descope's platform offloads the complexities of identity management, providing features like token validation, scope-based access control, and comprehensive audit trails for all agent activity.

## Architecture Overview

Our secure architecture consists of three core components that work in concert to provide a robust and auditable system for AI agent interactions:

1. **Claude Desktop (AI Agent):** The MCP client that initiates tool requests. It is configured with a long-lived Bearer token from Descope to authenticate itself with our secure gateway.
2. **lovable.dev (Next.js MCP Server):** This is the public-facing gateway generated by and deployed on lovable.dev. It hosts a Next.js application with a custom API route that acts as the MCP server. Its primary roles are:
   - Intercepting all incoming requests from the AI agent.
   - Using the Descope Node SDK to validate the Bearer token.
   - If the token is valid, it forwards the request to the appropriate n8n webhook, adding a secret API key for n8n to authenticate the request.
3. **n8n (Tool Backend):** Your n8n instance hosts the workflows that perform the actual tasks. Each tool is backed by a workflow triggered by a Webhook node, which is configured to only accept requests from our trusted lovable.dev server via a secret header.

The data flow is as follows: `Claude Desktop → lovable.dev Next.js Server (Descope Auth) → n8n Webhook`. This ensures that your n8n instance is never directly exposed to the public internet.

# Chapter 1: Setting up n8n Workflows as Tool Backends

First, we need to create the n8n workflows that will serve as the backend logic for our MCP tools. Each tool will correspond to a workflow triggered by a secure webhook.

### *Step 1: Create an n8n Workflow*

Start by creating a new workflow in your n8n instance. For this example, let's build a simple workflow that takes a text input and returns a "Hello, [text]!" message.

### *Step 2: Add and Configure the Webhook Node*

1. Drag a **Webhook** node onto the canvas. This will be the entry point for your tool.
2. In the Webhook node's parameters, set the **HTTP Method** to `POST`.
3. Under **Authentication**, select **Header Auth**. This is crucial for security.
4. In the credentials dropdown, select "Create New". Give your credential a name (e.g., "Lovable Server Auth"). Set the **Name** field to `X-N8N-API-KEY` and generate a strong, random string for the **Value**. Save this value securely; we will use it later in our lovable.dev environment variables.
5. Copy the **Test URL** from the Webhook node. You will need this URL for your lovable.dev server configuration.

### *Step 3: Add a "Respond to Webhook" Node*

To send a response back, connect a **Respond to Webhook** node to your workflow logic. You can use an expression to construct a JSON response. For example, to return the input data, you could use:

```
{{ JSON.stringify({ "message": "Hello, " + $json.body.name + "!" }) }}
```

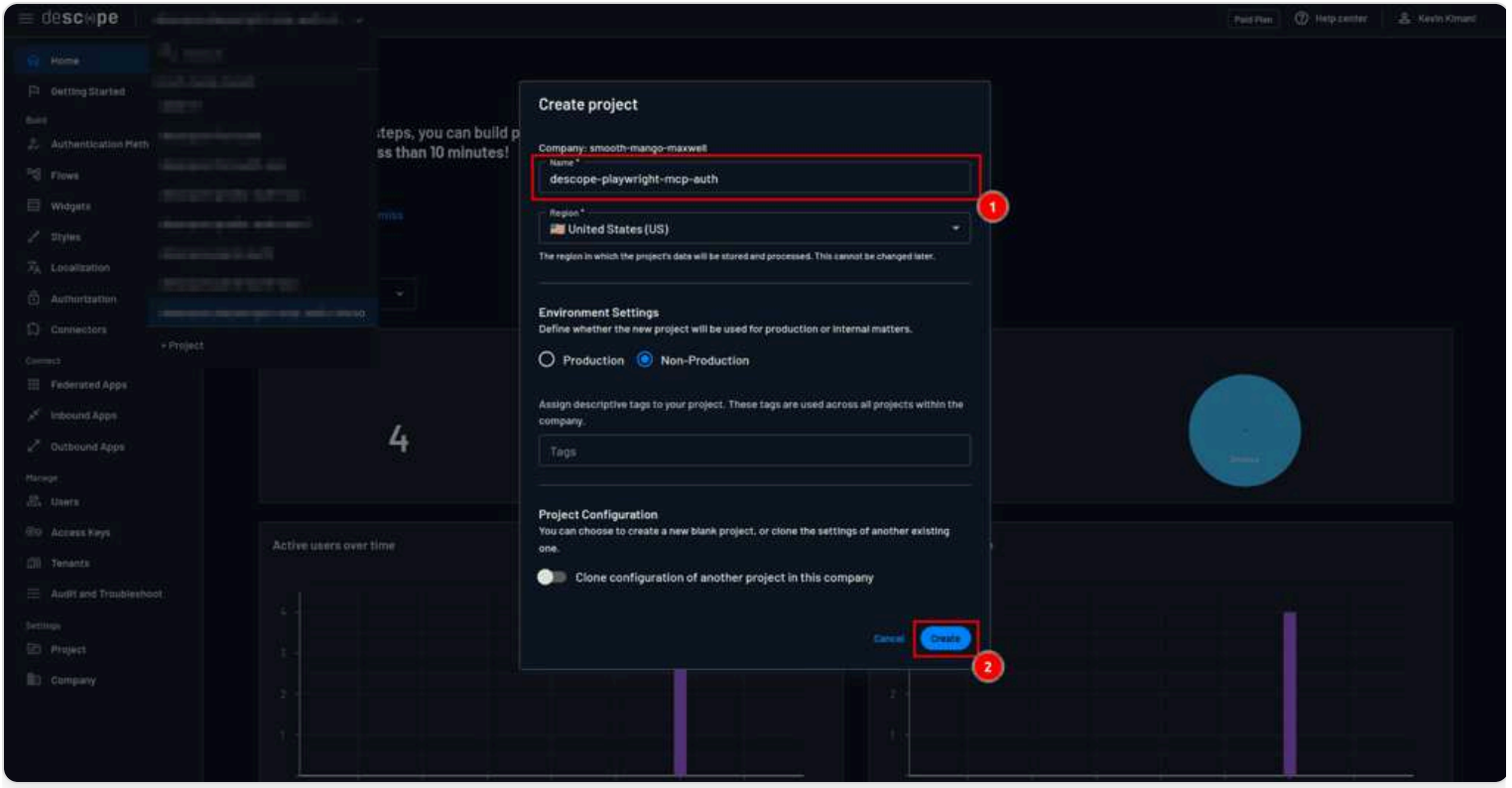
Save and activate your workflow. Your n8n tool backend is now ready and waiting for secure requests.

# Chapter 2: Configuring Descope's Agentic Identity Hub

Descope will act as the identity provider for our AI agent, issuing and validating the tokens that grant access to our MCP server.

### *Step 1: Create a Descope Project*

If you haven't already, sign up for a Descope account. Once logged in, create a new project from the dashboard. Note your **Project ID**, as you'll need it later.




*Create a new project in the Descope console.*

### *Step 2: Generate a Long-Lived Bearer Token for the Agent*

For a client like Claude Desktop, we'll use a long-lived access key (acting as a Bearer token) for simplicity.

1. Navigate to **Manage** > **Access Keys**.
2. Click **+ Access Key**.
3. Give the key a name (e.g., "Claude Desktop Agent Key").
4. You can associate it with a user and assign it specific roles for RBAC (covered in Chapter 6). For now, you can leave these blank.
5. Click **Create Key** and copy the generated token. **This is the Bearer token you will use in your Claude Desktop configuration.**

# Chapter 3: Building & Deploying the MCP Server with lovable.dev

This chapter is the core of our "vibe-coding" workflow. Instead of writing the Next.js server from scratch, we will instruct the lovable.dev AI to build it for us.

### *Step 1: Provide a Detailed Prompt to lovable.dev*

Log in to your lovable.dev account and start a new project. Provide a detailed prompt that outlines the entire application structure and logic. This empowers the AI to generate a near-complete solution.

**Detailed Sample Prompt for lovable.dev:**

```
Create a Next.js application that acts as a secure MCP (Model Context Protocol) server.

The application must:
1.  Use the App Router.
2.  Install the following dependencies: `@vercel/mcp-handler` and `@descope/node-sdk`.
3.  Create a dynamic API route at `src/app/api/mcp/[...transport]/route.ts` to handle both S
4.  This route must define an MCP tool named 'run_n8n_task'.
    - The tool should have a description like "Executes a specific n8n workflow."
    - The input schema should be a generic JSON object to pass data to n8n.
5.  The `execute` function for this tool must:
    - Read the n8n webhook URL from an environment variable `N8N_WEBHOOK_URL`.
    - Read a secret API key from an environment variable `N8N_API_KEY`.
    - Perform a `fetch` POST request to the `N8N_WEBHOOK_URL`.
    - Include the `Content-Type: application/json` header.
    - Include a security header `X-N8N-API-KEY` with the value from the `N8N_API_KEY` enviro
    - Forward the input from the tool as the JSON body of the fetch request.
    - Return the response from the n8n webhook.
6.  Implement security: The API route must be protected. Before processing any MCP request,
    - Create a `verifyToken` function that uses the Descope Node SDK (`@descope/node-sdk`).
    - The `verifyToken` function should initialize the Descope client with the `DESCOPE_PROJ
    - It will use `descope.validateSession(bearerToken)` to validate the token.
    - If the token is valid, map the claims to the MCP `AuthInfo` object. If invalid, deny a
7.  Use the `withMcpAuth` wrapper from `@vercel/mcp-handler` to protect the main handler, pa
```

### *Step 2: Configure Environment Variables in lovable.dev Cloud Settings*

Once lovable.dev generates the application, you need to provide the necessary secrets and configuration.

1. In your lovable.dev project, navigate to the **Cloud** tab or your project's deployment settings.
2. Find the section for **Environment Variables** or **Secrets**.
3. Add the following keys and their corresponding values:
   - `DESCOPE_PROJECT_ID`: Your Project ID from the Descope console.
   - `N8N_WEBHOOK_URL`: The full Test URL of the webhook from your n8n workflow (Chapter 1).
   - `N8N_API_KEY`: The secret value you created for the `X-N8N-API-KEY` header in your n8n webhook node (Chapter 1).
4. Save and redeploy the application. lovable.dev will automatically make these variables available to your Next.js application at runtime.

# Chapter 4: Under the Hood: Securing the API Route

The prompt in the previous chapter instructed lovable.dev to generate code that secures your API. Let's briefly examine how this works. The generated code will use a pattern similar to the following to wrap the core MCP logic with Descope authentication.

```
// A simplified view of the generated src/app/api/mcp/[...transport]/route.ts

import { createMcpHandler, withMcpAuth } from 'mcp-handler';
import { AuthInfo } from '@modelcontextprotocol/sdk/server/auth/types.js';
import DescopeClient from '@descope/node-sdk';

// ... tool definitions (run_n8n_task)

// Descope token verification function (generated by lovable.dev)
const verifyToken = async (
  req: Request,
  bearerToken?: string,
): Promise<AuthInfo | undefined> => {
  if (!bearerToken) {
    return undefined; // No token, no access
  }

  try {
    const descope = DescopeClient({
      projectId: process.env.DESCOPE_PROJECT_ID!,
    });
    const authInfo = await descope.validateSession(bearerToken);

    if (!authInfo.token.sub) {
        return undefined; // Invalid token
```

```
    }

    // Map Descope token claims to MCP AuthInfo
    return {
      token: bearerToken,
      scopes: authInfo.token.scopes || [],
      clientId: authInfo.token.sub, // Use subject as a unique ID for the agent
    };
  } catch (error) {
    console.error('Descope token validation failed:', error);
    return undefined;
  }
};

// Create the base handler with tools
const baseHandler = createMcpHandler({
  tools: [/* ...run_n8n_task tool... */],
});

// Wrap the handler with Descope authentication
const authHandler = withMcpAuth(baseHandler, verifyToken, {
  required: true, // All requests must be authenticated
});

export { authHandler as GET, authHandler as POST };
```

This code ensures that no request can reach your tool's `execute` logic without first passing a valid Descope-issued Bearer token, effectively securing your n8n backend.

# Chapter 5: Integrating with Claude Desktop Client

The final step is to configure the Claude Desktop client to communicate with your newly deployed and secured MCP server on lovable.dev.

### *Step 1: Locate `claude_desktop_config.json`*

Open the Claude Desktop settings to edit its configuration file. The file is located at:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

### *Step 2: Add the Server Configuration*

Add an entry for your lovable.dev MCP server. This configuration tells Claude the server's URL and how to authenticate using the Descope Bearer token.

```
{
  "mcpServers": {
    "lovable-n8n-secure": {
      "url": "https://your-lovable-app-url.lovable.dev/api/mcp",
      "authentication": {
        "type": "http_header",
        "name": "Authorization",
        "value": "Bearer YOUR_DESCOPE_ACCESS_KEY"
      }
    }
  }
}
```

Replace `https://your-lovable-app-url.lovable.dev/api/mcp` with the actual URL of your deployed lovable.dev application and `YOUR_DESCOPE_ACCESS_KEY` with the long-lived token you generated in Chapter 2. Restart Claude Desktop to apply the changes.

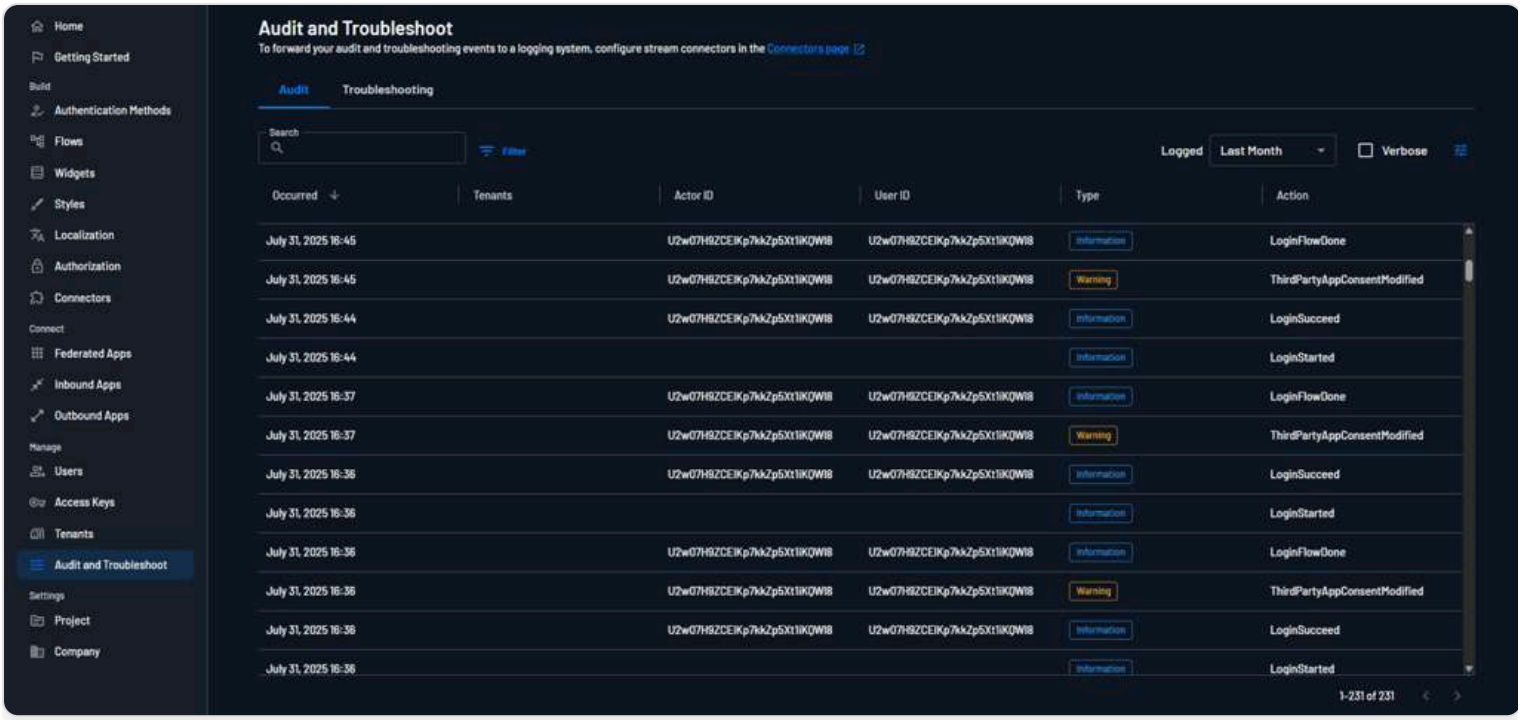
# Chapter 6: Security Best Practices & Auditing

With the core architecture in place, you can enhance security and monitoring using Descope's advanced features.

### *Monitoring Agent Activity with Audit Logs*

Descope provides a comprehensive audit trail for every authentication and token validation event.

1. In the Descope console, navigate to **Audit and Troubleshoot**.
2. Here, you can see every time the Claude agent's token was validated, including the timestamp, actor ID, and outcome.
3. This visibility is critical for monitoring agent activity, detecting anomalies, and troubleshooting access issues. You can even stream these logs to a SIEM platform for advanced analysis.




*Use Descope's audit logs to monitor every agent action and ensure security.*

### *Implementing Role-Based Access Control (RBAC)*

You can restrict specific tools to agents with certain roles. For example, you might have a high-risk n8n workflow for deleting data that only "Admin Agents" can access.

1. **In Descope:** Create a Role (e.g., "N8nAdmin") and assign it to the Access Key you generated for your agent.
2. **In your Next.js code:** Modify the `verifyToken` function (or prompt lovable.dev to do so) to check for this role in the JWT claims.

```
// Example RBAC check in verifyToken
const authInfo = await descope.validateSession(bearerToken);
const roles = authInfo.token.roles || [];

if (toolName === 'deleteDataWorkflow' && !roles.includes('N8nAdmin')) {
  throw new Error('Access denied: Admin role required.');
}
```

# Conclusion

By following this guide, you have constructed a secure, scalable, and production-ready architecture for exposing n8n workflows as MCP tools. You successfully leveraged the "vibe-coding" paradigm of

lovable.dev to generate a Next.js gateway that acts as a secure intermediary, using Descope's Agentic Identity Hub to offload the critical tasks of authentication, authorization, and auditing.

This end-to-end solution ensures that your powerful n8n automation capabilities are accessible to AI agents without compromising on security. Your n8n instance remains protected behind a secure proxy, and every agent interaction is authenticated, authorized, and logged, providing the enterprise-grade governance required for modern AI-powered systems.